

Technical Design Document (TTD)

Ledger & Credit-Debit Management System

Document Type	Technical Design Document (TTD)
Version	1.1
Date	June 22, 2026
Author	Jalaj Singhal (Lead Architect & Systems Engineer)
Target System	Jalajs Ledger Application (Python/SQLite/CustomTkinter)
Status	Approved & Released

Architectural Overview

The Ledger Management System uses a clean **Three-Tier Architecture** designed for robust segregation of concerns, simple database migrations, and isolated side-effect wrappers:

- **Presentation Layer (UI):** Built using CustomTkinter App Shells, Views, and custom widgets (DataTable, StatCard). Screens do not communicate directly with databases or perform business validations; they interact solely with the Service Layer.
- **Service Layer (Services):** Enforces business data rules, hashes passwords, performs XOR obfuscation, generates PDFs/Excel sheets, manages email/WhatsApp dispatches, and validates database transactions.
- **Repository/Data Layer (Repositories):** Abstracts database-specific querying. Utilizes BaseRepository wrappers and parameterized prepared statements to prevent SQL injections.
- **Database Layer (Database):** Connects to the local SQLite storage engine (ledger.db). Enforces Write-Ahead Logging (WAL) and index constraints to ensure data integrity during parallel writes.

1. Codebase Structure & Component Mapping

The codebase has a clean layout. The following is a detailed overview of the component files inside ledger_app/:

- main.py: The bootshell entry point that appends project roots, initializes settings, and boots the CustomTkinter App.
- database/schema.py: Handles SQLite database schema construction, pre-seeds data (default payment modes), and conducts index updates.
- models/: Contains data containers defining application entities such as Customer, Transaction, and PaymentMode.
- repositories/: Manages database transactions. customer_repository.py, transaction_repository.py, and payment_mode_repository.py implement localized SQL commands.
- services/pdf_security_service.py: Encapsulates PBKDF2 password hashes, password strength tests, XOR at-rest key obfuscations, and PyPDF2 AES bindings.
- services/export_service.py: Handles PDF (ReportLab Platypus, custom NumberedCanvas page numbering) and Excel exports (OpenPyXL width fitments and cell styling).
- services/whatsapp_service.py: Leverages Win32 clipboard functions and key simulation automation to control the native WhatsApp Desktop client.
- services/email_service.py: Transmits HTML closing updates and daily summary attachments via secure SMTP TLS/SSL tunnels.
- ui/theme.py: Stores global color tokens, styles, and custom font definitions for CustomTkinter.
- ui/screens/: Form layouts and screen controllers, including the dashboard, client directories, entry grids, reports, and settings.

2. Relational Database Schema Design

The SQLite database is stored locally inside ledger.db. It enforces transactional integrity, index constraints, and cascading deletes:

Table: customers

```
CREATE TABLE customers (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL,  
  phone TEXT,  
  address TEXT,  
  notes TEXT,  
  opening_balance REAL NOT NULL DEFAULT 0.0,  
  created_at TEXT NOT NULL DEFAULT (datetime('now', 'localtime'))  
);
```

Table: payment_modes

```
CREATE TABLE payment_modes (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  mode_name TEXT UNIQUE NOT NULL  
);
```

Table: transactions

```
CREATE TABLE transactions (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  customer_id INTEGER NOT NULL,  
  transaction_type TEXT NOT NULL CHECK(transaction_type IN ('CREDIT', 'DEBIT', 'DAILY_CHARGE')),  
  amount REAL NOT NULL CHECK(amount >= 0.0),  
  payment_mode_id INTEGER,  
  description TEXT,  
  transaction_date TEXT NOT NULL DEFAULT (datetime('now', 'localtime')),  
  FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE CASCADE,  
  FOREIGN KEY (payment_mode_id) REFERENCES payment_modes(id) ON DELETE SET NULL  
);
```

3. Core Technical Services Design

3.1 Cryptographic & Security Operations

Password Verification: The password complexity rules require a minimum of 8 characters, at least one uppercase letter, one lowercase letter, one digit, and one special character. User verification hashes rely on the NIST-approved PBKDF2-HMAC-SHA256 protocol, executing 100,000 iterations against a cryptographically random 16-byte salt. Secrets verification uses a constant-time comparison to prevent side-channel timing attacks.

At-Rest Password Protection: Sensitive local settings—specifically SMTP keys—are XOR-obfuscated using a dynamic keystream expanded from the configuration's salt bytes. While not a replacement for operating system keychains, this prevents accidental plain-text exposure in configuration settings.

PDF Document Protection: When active, the export engine post-processes output PDFs through a PyPDF2 wrapper that copies pages into a fresh writer, then applies AES-128 encryption using the user's password. The original unencrypted PDF is replaced atomically via `shutil.move`.

3.2 Advanced Document Layouts & Spreadsheet Logic

PDF Generation: Designed using ReportLab Platypus. Features clean grids, HSL-tailored card background Fills, and zebra striping. The custom canvas `NumberedCanvas` records layout parameters on the first pass and draws running footers, business headers, and dynamic page counts on the final draw pass.

Excel Generation: Implemented with OpenPyXL. Applies steel-blue column header Fills, thin borders, and zebra striping. The service dynamically evaluates data string lengths and auto-adjusts cell column widths to prevent data truncation.

3.3 Native Messaging Automations

SMTP Delivery: Sends closing summaries over TLS/SSL (ports 587/465). Documents are read in binary, wrapped in base64 MIME blocks, and appended as standard multipart attachments.

WhatsApp Desktop Automation: Invokes the native URI (`whatsapp://send?phone=...`) using `WScript.Shell`. Once focused via Win32 user32 focus APIs, the service places the PDF path or image onto the Windows clipboard and injects keystrokes (Ctrl+V, Enter) using native `SendKeys` scripts.

4. Mathematical Formulation & Concurrency

4.1 Running Balance Formulations

The mathematical formula to evaluate a customer's running balance chronologically is:

$$\begin{aligned}
 & \text{\textit{Running Balance}} = \text{\textit{Opening Balance}} + \sum \text{\textit{DEBIT}} + \sum \text{\textit{DAILY_CHARGE}} - \\
 & \sum \text{\textit{CREDIT}}
 \end{aligned}$$

Where Credits represent client payments (reducing the outstanding balance) and Debits/Charges represent sales on credit (increasing the balance).

4.2 Concurrency & Thread Isolation

To avoid freezes in CustomTkinter's single-threaded event loop, all network services (SMTP, WhatsApp automation, DB Backups) are executed on separate background daemon threads. This prevents the UI from becoming unresponsive during long-running tasks.

4.3 Installed Third-Party Library Dependencies

Below is the exact list of third-party Python library dependencies used by the Ledger Management System:

- customtkinter
- reportlab
- openpyxl
- pandas
- matplotlib
- pillow
- selenium
- tkcalendar
- babel
- pypdf2